

LAB 2

Writing Effective Test Cases

192-211 Automated Software Testing
International College, Siam University

Not All Tests Are Equal

You can write a hundred tests that catch nothing, or ten that catch every important bug. This lab teaches what separates a valuable test from a wasteful one.

The Qualities of a Good Test

Quality	Meaning
Focused	Tests one behavior
Independent	Does not depend on other tests
Repeatable	Same result every time
Readable	Name and body explain intent
Fast	Runs quickly

What You Will Learn

- Apply the AAA pattern rigorously
- Make tests independent of each other
- Find edge cases and boundary values
- Write test names that document behavior
- Balance positive and negative test cases

Environment Note

This lab assumes Python and pytest. We set up fresh at the start. Click **START** when you are ready!

STEP 1

The AAA Pattern in Depth

Set Up

```
apt-get update -qq && apt-get install -y python3 python3-pip python3-venv 2>/dev/null  
mkdir -p ~/testcase-lab && cd ~/testcase-lab  
python3 -m venv venv && source venv/bin/activate  
pip install pytest -q
```

Create Code to Test

```
cd ~/testcase-lab
cat > bank.py << 'PY'
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self.balance += amount
        return self.balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount
        return self.balance
```

PY

Write a Clear AAA Test

```
cd ~/testcase-lab
cat > test_bank.py << 'PY'
from bank import BankAccount

def test_deposit_increases_balance():
    account = BankAccount(balance=100)
    new_balance = account.deposit(50)
    assert new_balance == 150
PY
pytest test_bank.py -v
```

Why Separate the Three Phases?

Phase	Purpose
Arrange	Set up everything the test needs
Act	Perform the single action being tested
Assert	Verify exactly one outcome

Mixing these makes tests hard to read. Keeping them separate makes the intent obvious.

A Bad Test (What to Avoid)

```
cd ~/testcase-lab
cat > test_bad_example.py << 'PY'
from bank import BankAccount

def test_everything_at_once():
    account = BankAccount(100)
    account.deposit(50)
    account.withdraw(30)
    account.deposit(10)
    assert account.balance == 130
PY
```

A Bad Test (What to Avoid) (cont.)

This tests three actions at once. If it fails, which action broke? You cannot tell. Prefer one action per test.

Once your AAA test passes, move on!

STEP 2

Test Independence

The Problem: Shared State

```
cd ~/testcase-lab
cat > test_dependent.py << 'PY'
from bank import BankAccount

shared_account = BankAccount(100)

def test_a_deposit():
    shared_account.deposit(50)
    assert shared_account.balance == 150

def test_b_withdraw():
    shared_account.withdraw(30)
    assert shared_account.balance == 120
PY
pytest test_dependent.py -v
```

The Problem: Shared State (cont.)

If you run `test_b` alone, it fails - because it relied on `test_a`. This is fragile.

Prove the Problem

```
cd ~/testcase-lab && pytest test_dependent.py::test_b_withdraw -v
```

Running `test_b` alone fails! Bad tests hide bugs and create false failures.

The Fix: Each Test Owns Its Data

```
cd ~/testcase-lab
cat > test_independent.py << 'PY'
from bank import BankAccount

def test_deposit_independent():
    account = BankAccount(100)      # fresh account
    account.deposit(50)
    assert account.balance == 150

def test_withdraw_independent():
    account = BankAccount(100)      # fresh account
    account.withdraw(30)
    assert account.balance == 70
PY
pytest test_independent.py -v
```

Run Them in Any Order

```
cd ~/testcase-lab && pytest test_independent.py::test_withdraw_independent -v
```

Now each test passes on its own, in any order.

The Rule

Bad	Good
Global shared objects	Fresh setup per test
Test B needs Test A	Every test standalone
Order☐dependent	Order☐independent

Once your independent tests pass individually, move on!

STEP 3

Edge Cases and Boundary Values

Boundary Value Analysis

For any range, test:

- The minimum valid value
- The maximum valid value
- Just below minimum (invalid)
- Just above maximum (invalid)
- Zero and empty values

Create a Function with Boundaries

```
cd ~/testcase-lab
cat > grades.py << 'PY'
def letter_grade(score):
    if score < 0 or score > 100:
        raise ValueError("Score must be 0-100")
    if score >= 80:
        return "A"
    if score >= 70:
        return "B"
    if score >= 60:
        return "C"
    return "F"
```

PY

Test the Boundaries

```
cd ~/testcase-lab
cat > test_grades.py << 'PY'
import pytest
from grades import letter_grade

def test_boundary_a_grade():
    assert letter_grade(80) == "A"      # exactly at boundary
    assert letter_grade(79) == "B"      # just below

def test_boundary_pass_fail():
    assert letter_grade(60) == "C"      # lowest pass
    assert letter_grade(59) == "F"      # just failed

def test_minimum_valid():
    assert letter_grade(0) == "F"

def test_maximum_valid():
    assert letter_grade(100) == "A"

def test_below_minimum_invalid():
    with pytest.raises(ValueError):
        letter_grade(-1)
```

Why Boundaries Catch Bugs

A common bug is using `>` instead of `>=`. That bug only shows up at the exact boundary (score of 80). Testing 85 would miss it - testing 80 catches it.

The Boundary Checklist

Test	Value
Minimum	0
Just below min	-1
Maximum	100
Just above max	101
Each threshold	60, 70, 80
Just below threshold	59, 69, 79

Once all boundary tests pass, move on!

STEP 4

Descriptive Test Names

Bad vs Good Names

Bad	Good
test_1	test_deposit_increases_balance
test_account	test_withdraw_fails_when_insufficient_funds
test_works	test_empty_cart_has_zero_total

A Naming Convention

A useful pattern is: `test_[what]_[condition]_[expected]`

```
test_withdraw_when_insufficient_funds_raises_error
  |           |                               |
  what       condition                       expected result
```

Rewrite Tests with Clear Names

```
cd ~/testcase-lab
cat > test_named.py << 'PY'
import pytest
from bank import BankAccount

def test_deposit_positive_amount_increases_balance():
    account = BankAccount(100)
    account.deposit(50)
    assert account.balance == 150

def test_deposit_negative_amount_raises_value_error():
    account = BankAccount(100)
    with pytest.raises(ValueError):
        account.deposit(-10)

def test_withdraw_more_than_balance_raises_value_error():
    account = BankAccount(100)
    with pytest.raises(ValueError):
        account.withdraw(200)

def test_withdraw_exact_balance_leaves_zero():
    account = BankAccount(100)
```

Read the Output as Documentation

```
cd ~/testcase-lab && pytest test_named.py -v --no-header
```

Notice how the test names read like a specification of how the BankAccount should behave.
Good test names ARE documentation.

The Test Names as a Spec

Reading just the names tells you:

- Depositing a positive amount increases balance
- Depositing negative raises an error
- Withdrawing too much raises an error
- Withdrawing the exact balance leaves zero

Anyone can understand the rules without reading the code! Once your named tests pass, move on!

STEP 5

Positive and Negative Testing

Positive Tests: The Happy Path

```
cd ~/testcase-lab
cat > validators.py << 'PY'
import re

def validate_email(email):
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    if not re.match(pattern, email):
        raise ValueError(f"Invalid email: {email}")
    return True

def validate_age(age):
    if not isinstance(age, int):
        raise TypeError("Age must be an integer")
    if age < 0 or age > 150:
        raise ValueError("Age must be 0-150")
    return True

PY
```

Write Positive Tests

```
cd ~/testcase-lab
cat > test_positive.py << 'PY'
from validators import validate_email, validate_age

def test_valid_email_accepted():
    assert validate_email("student@siam.ac.th") is True

def test_valid_email_with_subdomain():
    assert validate_email("user@mail.example.com") is True

def test_valid_age_accepted():
    assert validate_age(25) is True

def test_boundary_ages_accepted():
    assert validate_age(0) is True
    assert validate_age(150) is True
PY
pytest test_positive.py -v
```

Write Negative Tests

```
cd ~/testcase-lab
cat > test_negative.py << 'PY'
import pytest
from validators import validate_email, validate_age

def test_email_without_at_rejected():
    with pytest.raises(ValueError):
        validate_email("notanemail.com")

def test_email_without_domain_rejected():
    with pytest.raises(ValueError):
        validate_email("user@")

def test_negative_age_rejected():
    with pytest.raises(ValueError):
        validate_age(-5)

def test_age_as_string_rejected():
    with pytest.raises(TypeError):
        validate_age("twenty")
PY
pytest test_negative.py -v
```

Run Both Suites Together

```
cd ~/testcase-lab && pytest test_positive.py test_negative.py -v
```

The Balance

Test Type	Question	Example
Positive	Does valid input work?	Valid email accepted
Negative	Is invalid input rejected?	Bad email raises error

Both matter. Code that accepts good input but crashes on bad input is not production-ready. Once both suites pass, you have completed the lab!

What You Accomplished

- Applied the AAA pattern rigorously
- Made tests independent of each other
- Tested edge cases and boundary values
- Wrote descriptive, self-documenting test names
- Balanced positive and negative test cases

The Principles of Good Tests

Principle	Why
One behavior per test	Failures pinpoint the problem
Independent	Order does not matter
Boundary coverage	Bugs hide at edges
Descriptive names	Tests document behavior
Positive + negative	Handle good AND bad input

Boundary Testing Checklist

For any range: test the min, max, just-below-min, just-above-max, and each internal threshold.

Review Questions

1. Why must tests be independent of each other?
2. A function accepts scores 0-100. What boundary values should you test?
3. What is the difference between positive and negative testing?

Coming Up Next

In Lab 3 you will learn advanced assertions and how to organize large test suites with classes and markers.